

CÓDIGO FUENTE ARDUINO IDE (ESP32)

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <TinyGPS++.h>
#include <HardwareSerial.h>

// ===== CONFIGURACIÓN WIFI Y BACKEND
// =====
const char* ssid = "MARCELIANO";
const char* password = "esp32prueba";
const char* backendURL = "https://proyecto-wally-backend.onrender.com/api/datos";
const char* deviceId = "ESP32_WALLY_01";

// ===== COORDENADAS POR DEFECTO (UNFV)
// =====
// Se usan cuando el GPS todavía no tiene fix válido.
const double LAT_DEFAULT = -12.04575409384512;
const double LNG_DEFAULT = -77.04798170792918;

// ===== PINES =====
#define MQ7_PIN 34
#define MQ135_PIN 35

HardwareSerial gpsSerial(2);
TinyGPSPlus gps;

// ===== CALIBRACIÓN =====
float Ro_MQ7 = 0.72;
float Ro_MQ135 = 1.8;
float baselineCO = 0;

// ===== INTERVALOS =====
unsigned long ultimoEnvio = 0;
const unsigned long intervaloEnvio = 1000; // 1 segundo

unsigned long ultimoEstadoGPS = 0;
const unsigned long intervaloEstadoGPS = 1000;

unsigned long inicioPrograma = 0;

void setup() {
  Serial.begin(115200);
  delay(1000);
  gpsSerial.begin(9600, SERIAL_8N1, 16, 17);

  analogReadResolution(12);
```

```

inicioPrograma = millis();

Serial.println("\n=== WALLY - INICIO ===\n");

conectarWiFi();

Serial.println("Calibrando baseline de CO (20s, no muevas nada)...");
delay(20000);

float sumaCO = 0;
for (int i = 0; i < 20; i++) {
    sumaCO += analogRead(MQ7_PIN);
    delay(200);
}
float promedioCO = sumaCO / 20;
baselineCO = calcularPPM_MQ7(promedioCO);
Serial.printf("Baseline CO establecido: %.2f ppm\n\n", baselineCO);

calibrarRoMQ135();

Serial.println("\n=== CALIBRACION COMPLETA - INICIANDO ===\n");
Serial.println("[GPS] Buscando fix. Mientras tanto se usan coordenadas por defecto (UNFV)...\n");
}

void loop() {
    while (gpsSerial.available() > 0) {
        gps.encode(gpsSerial.read());
    }

    if (millis() - ultimoEstadoGPS >= intervaloEstadoGPS) {
        ultimoEstadoGPS = millis();
        mostrarEstadoGPS();
    }

    if (millis() - ultimoEnvio >= intervaloEnvio) {
        ultimoEnvio = millis();

        int adcCO = analogRead(MQ7_PIN);
        int adcMQ135 = analogRead(MQ135_PIN);
        float co = leerCO();
        float mq135 = leerMQ135();

        // Si el GPS no tiene fix válido, se usan las coordenadas por defecto
        // (UNFV) en vez de 0.0, 0.0.
        double lat = gps.location.isValid() ? gps.location.lat() : LAT_DEFAULT;
        double lng = gps.location.isValid() ? gps.location.lng() : LNG_DEFAULT;
    }
}

```

```

    Serial.println("=== Lectura ===");
    Serial.printf("CO: %.2f ppm (raw %d) | MQ135: %.2f ppm (raw %d)\n",
        co, adcCO, mq135, adcMQ135);
    Serial.printf("GPS: lat=%.6f, lng=%.6f (fix=%s)\n", lat, lng, gps.location.isValid() ? "si" :
        "no, usando default UNFV");

    enviarDatos(co, mq135, adcCO, adcMQ135, lat, lng);
}
}

// ===== GPS - DIAGNOSTICO CON DATOS CRUDOS
=====
void mostrarEstadoGPS() {
    unsigned long segundosCorriendo = (millis() - inicioPrograma) / 1000;

    Serial.printf("[GPS %lus] Chars procesados: %lu | Sentencias validas: %lu | Satelites: %s",
        segundosCorriendo,
        gps.charsProcessed(),
        gps.passedChecksum(),
        gps.satellites.isValid() ? String(gps.satellites.value()).c_str() : "sin dato"
    );

    if (gps.location.isValid()) {
        Serial.printf(" | FIX OK -> lat=%.6f lng=%.6f\n", gps.location.lat(), gps.location.lng());
    } else if (gps.charsProcessed() < 10) {
        Serial.println(" | SIN CONEXION FISICA - revisa cables TX/RX/VCC/GND (usando default
        UNFV)");
    } else {
        Serial.println(" | Buscando fix... (usando default UNFV mientras tanto)");
    }
}

// ===== WIFI =====
void conectarWiFi() {
    WiFi.disconnect(true);
    delay(100);
    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid, password);

    Serial.print("Conectando a WiFi");
    int intentos = 0;
    while (WiFi.status() != WL_CONNECTED && intentos < 20) {
        delay(500);
        Serial.print(".");
        intentos++;
    }

    if (WiFi.status() == WL_CONNECTED) {

```

```

    Serial.println("\nWiFi conectado. IP: " + WiFi.localIP().toString());
} else {
    Serial.println("\nNo se pudo conectar a WiFi. Reintentando en el loop...");
}
}

// ===== CALIBRACIÓN MQ-135 =====
void calibrarRoMQ135() {
    Serial.println("Calibrando Ro de MQ-135 (20s en aire limpio)...");
    delay(20000);

    float suma = 0;
    for (int i = 0; i < 20; i++) {
        suma += analogRead(MQ135_PIN);
        delay(200);
    }
    float promedioADC = suma / 20;
    float voltaje = promedioADC * (3.3 / 4095.0);
    float RS = (3.3 - voltaje) / voltaje * 10.0;
    Ro_MQ135 = RS / 3.6;

    Serial.printf("MQ-135 -> Ro calculado: %.3f\n", Ro_MQ135);
}

// ===== MQ-7 (CO) =====
float calcularPPM_MQ7(float adcValue) {
    if (adcValue >= 4090) {
        return 9999.0;
    }

    float voltaje = adcValue * (3.3 / 4095.0);
    if (voltaje < 0.01) voltaje = 0.01;

    float RS = (3.3 - voltaje) / voltaje * 10.0;
    float ratio = RS / Ro_MQ7;
    float ppm = 100.0 * pow(ratio, -1.5);

    if (ppm > 9999.0) ppm = 9999.0;

    return ppm;
}

float leerCO() {
    int adcValue = analogRead(MQ7_PIN);
    float ppm = calcularPPM_MQ7(adcValue);

    if (ppm < 0.0) ppm = 0.0;
    if (ppm > 60.0) ppm = 60.0;
}

```

```

    return ppm;
}

// ===== MQ-135 =====
float leerMQ135() {
    int adcValue = analogRead(MQ135_PIN);
    float voltaje = adcValue * (3.3 / 4095.0);
    if (voltaje < 0.01) voltaje = 0.01;

    float RS = (3.3 - voltaje) / voltaje * 10.0;
    float ratio = RS / Ro_MQ135;
    float ppm = 116.6 * pow(ratio, -2.769);

    float ppmEscalado = ppm * 45.0 + 400.0;

    if (ppmEscalado < 350) ppmEscalado = 350;
    if (ppmEscalado > 1500) ppmEscalado = 1500;

    return ppmEscalado;
}

// ===== ENVÍO AL BACKEND =====
void enviarDatos(float co, float mq135, int co_raw, int mq135_raw, double lat, double lng) {
    if (WiFi.status() != WL_CONNECTED) {
        Serial.println("WiFi desconectado, intentando reconectar...");
        conectarWiFi();
        if (WiFi.status() != WL_CONNECTED) return;
    }

    HTTPClient http;
    http.begin(backendURL);
    http.addHeader("Content-Type", "application/json");

    String payload = "{";
    payload += "\"device_id\": \"" + String(deviceId) + "\", ";
    payload += "\"co\": " + String(co, 2) + ", ";
    payload += "\"mq135\": " + String(mq135, 2) + ", ";
    payload += "\"co_raw\": " + String(co_raw) + ", ";
    payload += "\"mq135_raw\": " + String(mq135_raw) + ", ";
    payload += "\"lat\": " + String(lat, 6) + ", ";
    payload += "\"lng\": " + String(lng, 6);
    payload += "}";

    Serial.println("Enviando: " + payload);

    int codigoRespuesta = http.POST(payload);

```

```
if (codigoRespuesta > 0) {  
    String respuesta = http.getString();  
    Serial.printf("Respuesta HTTP %d: %s\n", codigoRespuesta, respuesta.c_str());  
} else {  
    Serial.printf("Error al enviar datos: %s\n", http.errorToString(codigoRespuesta).c_str());  
}  
  
http.end();  
}  
:
```